

HOCHSCHULE ESSLINGEN
Fakultät Informatik und Informationstechnik

Sommersemester 2026

PROJEKTDOKUMENTATION
im Rahmen der Vorlesung Datenbanken 2 (Laborprojekt)

Dozent: Prof. Dr. D. Hesse

GustoPantry

Intelligentes Web-basiertes Vorrats- und Rezeptmanagement mit
transaktionssicherer MS-SQL-Datenbanklogik

1. Projektsteckbrief

Entwickler:	Daniel Trbara (Einzelarbeit)
Studiengang:	Software Engineering / Medieninformatik
Entwicklungsumgebung:	Visual Studio Code, .NET Core 8.0 MVC
Datenbank-Server:	Microsoft SQL Server (Host: itnt0005, DB: SWB4_DB2_Projekt)
Frontend-Stack:	HTML5, Razor-Views, Tailwind CSS Framework
Logikfunktionen:	1 Trigger, 1 Stored Procedure, 1 Scalar Function

1 Projektbeschreibung

1.1 Fachlicher Hintergrund und Zielsetzung

Im modernen Haushaltsmanagement stellt die effiziente Überwachung von Lebensmittelbeständen eine Kernherausforderung dar. Mangelnde Übersicht und unbemerktes Ablaufen von Mindesthaltbarkeitsdaten begünstigen Lebensmittelverschwendung. **GustoPantry** löst diese Defizite durch eine dynamische Verknüpfung einer Vorratskammer mit einer relationalen Rezeptdatenbank. Das System zeichnet sich durch Flexibilität aus: Beim Kochvorgang können Mengen ad hoc angepasst, Einheiten (z.B. g in Stk) direkt überschrieben und Zutaten spontan ausgetauscht werden.

1.2 Komponenten und Benutzeroberfläche (GUI)

Die zentrale Schaltzentrale bildet die Vorratskammer. Hier werden Bestände über ein dynamisches Formular eingebunden. Die visuelle Kennzeichnung des Frischestatus erfolgt automatisiert über eine datenbankseitige Skalarfunktion, die den Ampelstatus (ROT, GELB, GRÜN) berechnet.

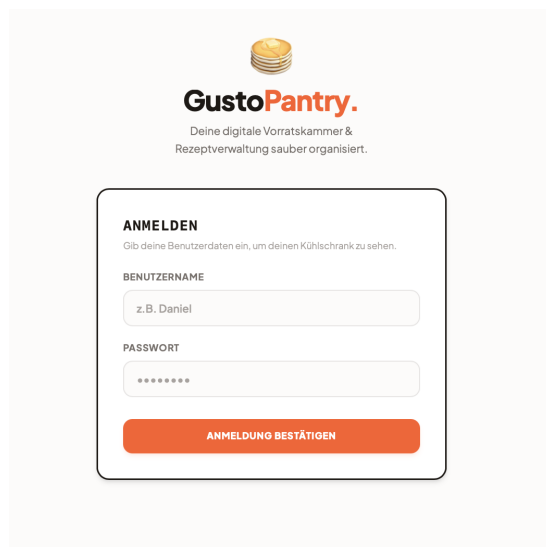


Abbildung 1: Sichere Login-Maske.

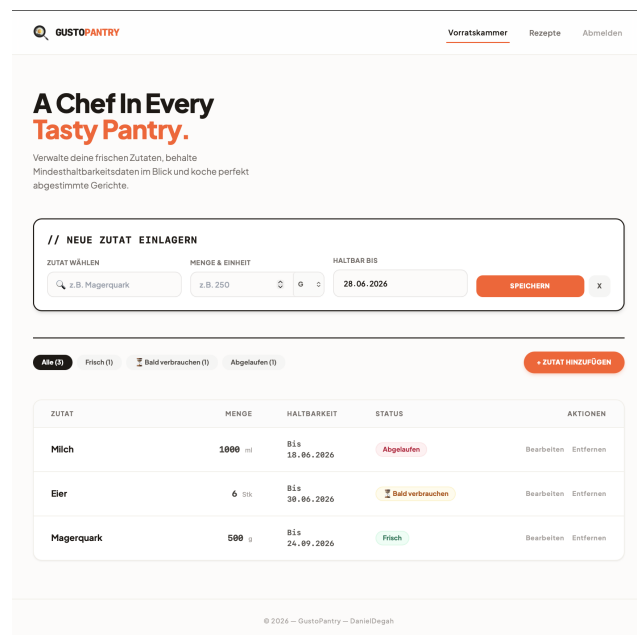


Abbildung 2: Vorratskammer mit MHD-Ampel.

Das Kochbuch stellt alle gespeicherten Rezepte dar. Über das Formular lassen sich neue Gerichte erfassen, wobei Zutatenzeilen dynamisch im Browser per JavaScript angefügt werden.

Inspirational Gusto Cookbook.
Stöbere durch deine Rezepte oder füge deiner digitalen Sammlung neue Kreationen hinzu.

// NEUES REZEPT EINTRAGEN
Erweitere dein Kochbuch um ein neues Gericht mit samt seinen Standard-Zutaten.

REZEPTNAME: z.B. Pancakes

STANDARD-ZUTATEN: Zutat (z.B. Mehl) Menge (g/s) X

ZUBEREITUNG / ANLEITUNG: 1. Zutaten mischen... 2. In die Pfanne geben...

REZEPT IM KOCHBUCH SPEICHERN

Curry #4
// BENÖTIGTE ZUTATEN
Currypulver 100 g
Reis 500 Stk
Wasser 1000 ml
// ZUBEREITUNG
Alles zusammen mischen und genießen

Einfacher Quark-Snack #3
// BENÖTIGTE ZUTATEN
Magerquark 250 g
Haferflocken 20 g
// ZUBEREITUNG
1. Den Magerquark in eine Schüssel geben.
2. Einen Schuss Wasser dazugeben und cremig schlagen.
3. Nach Belieben mit Haferflocken garnieren.

Abbildung 3: Rezept-Übersicht.

// NEUES REZEPT EINTRAGEN
Erweitere dein Kochbuch um ein neues Gericht mit samt seinen Standard-Zutaten.

REZEPTNAME: z.B. Pancakes

STANDARD-ZUTATEN: Zutat (z.B. Mehl) Menge (g/s) X

ZUBEREITUNG / ANLEITUNG: 1. Zutaten mischen... 2. In die Pfanne geben...

REZEPT IM KOCHBUCH SPEICHERN

Abbildung 4: Formular zur Rezepterschaffung.

Wählt der Anwender ein Gericht aus, öffnet sich die Detailansicht, gefolgt von dem dedizierten **Koch-Modal**. Hier wird der reelle Ist-Verbrauch erfasst und manipuliert, bevor er an die relationale Datenbank übergeben wird.

Einfacher Quark-Snack #3

// BENÖTIGTE ZUTATEN
Magerquark 250 g
Haferflocken 20 g

// ZUBEREITUNG
1. Den Magerquark in eine Schüssel geben.
2. Einen Schuss Wasser dazugeben und cremig schlagen.
3. Nach Belieben mit Haferflocken garnieren.

GERICHT JETZT KOCHEN

Abbildung 5: Einzelansicht eines Rezepts.

Einfacher Quark-Snack kochen

Passen die tatsächlich verbrauchten Mengen an, falls du Zutaten ersetzt oder reduziert hast!

Magerquark
Standard: 250 g

Haferflocken
Standard: 20 g

ABBRECHEN VERBRAUCH BUCHEN

Abbildung 6: Koch-Modal zur Verbrauchsanpassung.

2 Relationaler Datenbankentwurf (Schema)

Für die persistente Datenhaltung der Anwendung auf dem MS-SQL Server wurde ein streng normalisiertes Schema entworfen. Um Redundanzen zu vermeiden und referenzielle Integrität über Fremdschlüsselbeziehungen zu garantieren, wurden sieben Tabellen definiert:

1. **datrit02_Users**: Bildet die Grundlage der Benutzerverwaltung. Sie speichert die eindeutige **UserID** als Primärschlüssel sowie den **Username** und das zugehörige Passwort für die Session-Authentifizierung.
2. **datrit02_Ingredients**: Fungiert als globaler Stammdatenspeicher für alle im System registrierten Lebensmittel. Jede Zutat erhält einen eindeutigen Namen, um Mehrfacheinträge zu verhindern.
3. **datrit02_Pantry**: Verwaltet die physischen Bestände der Benutzer. Sie verknüpft **UserID** und **IngredientID** und speichert die exakte Menge (**Amount**) sowie das Mindesthaltbarkeitsdatum (**ExpirationDate**). Jede Charge wird isoliert betrachtet, um ein chronologisches Ablaufen zu ermöglichen.
4. **datrit02_Recipes**: Enthält die Stammdaten der Gerichte. Gespeichert werden der Rezeptname und eine ausführliche Zubereitungsanleitung (**Description**) als Freitext.
5. **datrit02_RecipeIngredients**: Realisiert die relationale $M : N$ -Beziehung zwischen Rezepten und Zutaten. Sie definiert über einen zusammengesetzten Primärschlüssel aus **RecipeID** und **IngredientID**, welche Standard-Mengen (**DefaultAmount**) für ein Gericht benötigt werden.
6. **datrit02_CookedHistory**: Protokolliert als Verlaufstabelle jeden durchgeführten Kochvorgang eines Benutzers mitsamt einem zeitgenauen Zeitstempel (**CookedAt**).
7. **datrit02_CookedIngredients**: Speichert die tatsächlich beim Kochen verbrauchten realen Mengen (**ActualAmount**). Diese Tabelle dient als direkter Auslöser für den nachfolgenden Datenbank-Trigger.

3 Analyse der serverseitigen Datenbanklogik

Um die Kriterien einer datenbankzentrierten Architektur zu erfüllen, wurden drei fundamentale Logikfunktionen direkt auf dem MS-SQL Server implementiert:

3.1 Logikfunktion 1: FEFO Bestands-Abzugs-Trigger

Der Trigger `datrit02_tr_ReducePantryOnCook` reagiert vollautomatisch `AFTER INSERT` auf die Tabelle `datrit02_CookedIngredients`. Er verwendet einen primären Cursor (`inserted_cursor`), um alle neu hinzugefügten Verbrauchszeilen sequenziell zu verarbeiten.

Innerhalb dieser Schleife öffnet sich ein verschachtelter sekundärer Cursor (`pantry_cursor`). Dieser selektiert die Bestände des betroffenen Benutzers für die jeweilige Zutat und sortiert sie strikt aufsteigend nach dem Ablaufdatum (`ORDER BY ExpirationDate ASC`). Die Schleife bucht die Mengen so lange chronologisch ab, bis der reelle Verbrauch vollständig gedeckt ist. Bestände mit leerer Menge werden über ein `DELETE` restlos entfernt, Teilmengen über ein `UPDATE` reduziert.

3.2 Logikfunktion 2: Stored Procedure zum Einlagern

Die Prozedur `datrit02_sp_AddIngredientToPantry` kapselt die Logik für die Bestandszufuhr. Um eine Fragmentierung der Datenbank zu verhindern, prüft sie mittels einer `IF EXISTS`-Klausel, ob für den Benutzer bereits ein identisches Lebensmittel mit exakt demselben Mindesthaltbarkeitsdatum im Schrank existiert. Wenn ja, wird die neue Menge addiert (`UPDATE`). Nur wenn kein exakter Match vorliegt, wird eine neue Zeile eingefügt (`INSERT`).

3.3 Logikfunktion 3: Skalarfunktion zur MHD-Ampelberechnung

Die Funktion `datrit02_fn_GetExpirationStatus` berechnet zur Laufzeit die verbleibende Haltbarkeit eines Lebensmittels. Über die SQL-Funktion `DATEDIFF` wird die mathematische Differenz in Tagen zwischen dem aktuellen Datum und dem Ablaufdatum ermittelt. Liegt der Wert unter Null, wird der Status `ROT` (Abgelaufen) zurückgegeben. Bei bis zu 3 Tagen Restlaufzeit schlägt die Ampel auf `GELB` (Bald verbrauchen) an, andernfalls bleibt sie auf `GRÜN` (Frisch).

4 Architektonische Schwierigkeiten und Lösungen

4.1 Konventionelles Routing und Namespace-Konflikte

Während der Initialisierung des .NET Core MVC-Projekts traten gravierende Routing-Fehler (HTTP 404) beim Aufruf des `RecipeController` auf. Die Ursachenanalyse ergab einen Konflikt zwischen der physikalischen Ordnerstruktur der Projektdatei (`src.csproj`) und den im C#-Code deklarierten Namespaces. Da .NET Core MVC standardmäßig konventionelles Routing nutzt, scheiterte die Zuordnung. Durch die Bereinigung des Root-Namespace einheitlich auf `src.*` und den consequenten Einsatz nativer Razor TagHelper konnte das Problem behoben werden.

4.2 Asynchrone Formulardaten und Model Binding

Beim dynamischen Ändern von Zutaten im JavaScript-Modal wurden die Formulardaten initial über lose, parallele Arrays an das Backend übertragen. Dies führte beim Model Binding im Controller zu massiven Konsistenzproblemen, da die Array-Indizes asynchron verarbeitet wurden und es zu Fehlabbuchungen kam. Die Lösung bestand in der Kapselung der Datenstrukturen in einem dedizierten Command-Objekt (`CookRecipeCommand`), welches eine feste Liste von komplexen Unterobjekten (`CookedIngredientInput`) erwartet. Durch diese explizite Typisierung parst das Framework die Daten streng objektorientiert.

4.3 Transaktionssicherheit und relationale Integrität

Da der Kochprozess mehrere voneinander abhängige Tabellenoperationen erfordert (Erstellen der Historie, Überprüfung der Stammdaten, Verbrauchs-Protokollierung), bestand bei Verbindungsabbrüchen das Risiko von korrupten Teilzuständen. Dieses Problem wurde im C#-Service durch die Kapselung des gesamten Blocks in einer atomaren Datenbanktransaktion (`conn.BeginTransaction()`) gelöst. Tritt an irgendeiner Stelle ein Fehler auf, wird über ein `transaction.Rollback()` der Ursprungszustand wiederhergestellt.

5 Fazit

Mit der Fertigstellung von **GustoPantry** wurde ein funktional hochgradig flexibles System zur Haushaltsorganisation realisiert. Das Zusammenspiel zwischen einer modernen objektorientierten Web-Infrastruktur im Backend und mächtiger, automatisierter Logik auf dem MS-SQL Server zeigt eindrucksvoll, wie moderne Full-Stack-Anwendungen von einer intelligenten Verteilung der Rechenlast profitieren können. Die Anwendung läuft stabil und verhindert durch die FEFO-gesteuerte Verbrauchslogik der Datenbank effektiv Lebensmittelverschwendung im Bestand.

6 Lessons Learned

1. **Mächtigkeit von DB-Triggern:** Die Auslagerung der Bestandsreduktion auf einen datenbankseitigen Trigger ist der Berechnung im Applikationscode vorzuziehen. Der Trigger schützt die Datenintegrität auf unterster Ebene, unabhängig davon, ob Daten über die GUI oder administrative SQL-Skripte manipuliert werden.
2. **Explizites Model Binding:** Lose Arrays sind im HTTP-Kontext hochgradig fehleranfällig. Ein strukturiertes Transport- und Command-Modell erhöht die Robustheit des Codes signifikant, da es Typensicherheit garantiert und Indizierungsfehler zur Laufzeit ausschließt.
3. **Datenkonsistenz durch Transaktionen:** Bei komplexen, mehrstufigen Schreibvorgängen auf einer relationalen Datenbank ist der Einsatz von Transaktionen unverzichtbar. Sie garantieren die Einhaltung der ACID-Kriterien (Atomarität, Konsistenz, Isolation, Dauerhaftigkeit) und verhindern korrupte Teilzustände bei abrupten Systemabbrüchen.